

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №5
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил преподаватель кафедры ЭВМ

_____/Баташев П.А./

Киров

2026

Цель работы

Освоить методы обработки данных, превышающих объем оперативной памяти, путем реализации алгоритмов внешней сортировки. Получить навыки работы с текстовыми и структурированными файлами в языках C++ и Python, провести сравнительный анализ

Постановка задачи

Генерируется файл *.csv содержащий поля nickname (строка), uuid (фикс. строка 36 символов), reg_date (фикс. строка 10 символов), level (число int - 4 байта), hours (число float - 4 байта), vac_ban (boolean значение - 1 байт).

Необходимо отсортировать файл используя не более 10% (100 МБ) ОЗУ и не дольше 10 минут по заданному полю и направлению сортировки.

В приложении с UI необходимо предоставить возможность выбора файла, генерации данных, сортировки с выбором модуля и отображением строк из обоих вайлов.

Описание алгоритма

Фаза сортировки

- Чтение входного файла блоками по 10 МБ (с откатом до конца строки).
- Парсинг строк CSV в массив записей.
- При заполнении массива записей – сортировка с помощью заданного компаратора (поле + направление).
- Запись отсортированного блока в бинарный временный файл.
- Повтор, пока не закончится входной файл.
- Последний неполный блок также сортируется и сохраняется.

Фаза слияния

- Открытие всех временных файлов, чтение из каждого первой записи.
- Помещение записей в приоритетную очередь с инвертированным компаратором.
- Цикл: извлечение минимальной записи → сериализация в текстовый CSV-буфер → если буфер почти заполнен – сброс в выходной файл.
- Чтение следующей записи из того же временного файла и добавление в очередь.
- После опустошения очереди – сброс остатка буфера.

Ссылка на репозиторий

<https://github.com/GIKExe/Subject/tree/main/Proga/laba5>

Анализ производительности: C++ vs Python

Общее описание алгоритма

Обе реализации выполняют внешнюю сортировку большого CSV-файла с фиксированной структурой записи (81 байт в бинарном представлении). Этапы:

1. Разбиение – чтение исходного файла блоками (10 МБ), парсинг CSV в бинарные записи, сортировка блока (до 500 000 записей) и запись во временный файл.
2. Слияние – K-way слияние временных файлов с использованием приоритетной очереди, формирование итогового отсортированного CSV-файла.

Наблюдаемые различия

Характеристика	C++	Python (numpy)
Скорость работы	Высокая (нативный код)	Заметно ниже (интерпретатор)
Потребление ОЗУ	~ 30 МБ (основной массив) + буфер (~ 10 МБ)	~ 40 МБ (numpy array) + буфер (~ 10 МБ)
Буферизация	10 МБ на чтение 10 МБ на запись	Аналогично 10 МБ

Ключевой вывод:

Python медленнее в 5–10 раз (зависит от данных), но потребление оперативной памяти практически идентично.

Детальное объяснение причин

1. Почему Python медленнее, но память такая же?

(a) Основная память – компактные структуры данных

- В C++ – массив структур Record (упакован, 81 байт на запись).
- В Python – numpy.ndarray с заданным dtype (также плотное хранение, без объектов-обёрток).
- Оба подхода дают примерно одинаковые затраты на хранение сортируемых записей (30 МБ для 500 000 записей).
- Дополнительные буферы (чтение/запись) – байтовые массивы по 10 МБ – тоже совпадают.

(b) Скорость – узкие места в Python

- Парсинг CSV (фаза разбиения)
C++: цикл по буферу, непосредственный доступ к символам, вызовы strtol/strtod (быстрые, библиотечные).

Python: поиск запятых `find()` и переводов строк, извлечение срезов `bytes`, преобразование `int()/float()` – каждая операция выполняется интерпретатором, что значительно медленнее.

- Сравнение записей при сортировке

C++: `std::sort` с компаратором, который инлайнится и работает с примитивами.

Python: `numpy.sort` – сама сортировка выполняется в C, но для структурированных массивов требуется сравнение полей, что создаёт накладные расходы на преобразование типов.

- Слияние (фаза слияния)

C++: чтение бинарной записи → распаковка полей (прямой доступ к памяти) → `sprintf` в буфер – всё в нативном коде.

Python: чтение 81 байта → `split(b'x00')` для обрезки строк → `struct.unpack` → форматирование через `%` (байтовое). Каждый `struct.unpack` и создание новой строки – дорого.

- Работа с приоритетной очередью

C++: `priority_queue` с элементами-структурами, операции сравнения – без аллокаций.

Python: `heapq` работает с объектами `MergeNode`. Создание объекта, извлечение ключа, сравнение – накладные расходы на динамическую типизацию.

- Отсутствие JIT-компиляции

CPython интерпретирует байт-код, каждый цикл и вызов функции имеют существенные издержки. Даже оптимизированный код на Python остаётся медленнее C++ в задачах обработки больших объёмов данных.

2. Влияние буферизации – сходство подходов

Обе реализации используют одинаковую стратегию буферизации:

- Буфер чтения 10 МБ – заполняется из файла, затем ищется последний перевод строки (чтобы не разорвать CSV-запись), курсор откатывается.
- Буфер записи 10 МБ – накопление строк сбросом при превышении порога (`WRITER_SIZE` - 1000).

Это позволяет эффективно работать с произвольным текстовым CSV-файлом, но:

- В C++ операции с буфером выполняются на уровне указателей и `memmove/memcmp`.
- В Python – поиск подстроки в `bytearray` и манипуляции срезами – медленнее, но не критично по сравнению с парсингом.

Таким образом, буферизация вносит примерно одинаковый относительный вклад накладных расходов, но абсолютная скорость в C++ выше за счёт языка.

3. Особенности языков, повлиявшие на реализацию

Аспект	C++	Python
Управление памятью	Ручное (<code>new/delete</code>) – минимальный оверхед	GC, но использовано <code>numru</code> и байтовые буфера, сборщик почти не вмешивается
Типизация	Статическая, всё известно на этапе компиляции	Динамическая, даже байтовые операции требуют проверок
Доступ к данным	Прямой через указатели, выравнивание <code>#pragma pack</code>	Через методы объектов, обёртки <code>bytes</code>
Оптимизация	Компилятор (<code>-O2</code> , <code>-O3</code>) агрессивно оптимизирует циклы	Интерпретатор не оптимизирует; использование встроенных функций (<code>numru</code>) частично спасает
Форматирование	<code>sprintf</code> (или более быстрый <code>std::to_chars</code>)	% с байтовыми строками – медленно из-за копирования и кодирования

Выводы

В ходе выполнения лабораторной работы я реализовал алгоритм внешней сортировки для CSV-файла, объём которого превышает доступную оперативную память (ограничение 100 МБ). Были разработаны две версии программы – на C++ и Python – с единым принципом работы: разбиение исходного файла на отсортированные блоки по 10 МБ с последующим K-образным слиянием через приоритетную очередь.

Основные результаты:

- Обеспечена сортировка файла произвольного размера в рамках заданных ограничений по памяти (до 100 МБ) и времени (менее 10 минут).
- Реализована буферизованная обработка CSV-потоков с корректной обработкой границ записей.
- Проведён сравнительный анализ производительности языков C++ и Python на идентичных входных данных.

Ключевые наблюдения:

- C++ показывает значительно более высокую скорость (в 5–10 раз быстрее) за счёт нативного кода, отсутствия накладных расходов на интерпретацию и эффективной работы с памятью (указатели, инлайнинг компараторов, быстрый парсинг через `strtol/strtod`).
- Потребление оперативной памяти в обеих реализациях практически одинаково (30–40 МБ для массива записей + буферы) благодаря использованию в Python плотных `numpy.ndarray` вместо объектов-обёрток.
- Узкими местами в Python являются парсинг CSV, распаковка бинарных структур через `struct.unpack`, форматирование строк и динамическая типизация в приоритетной очереди.

В результате работы я освоил методы внешней сортировки, приобрел практические навыки работы с большими текстовыми и бинарными файлами в C++ и Python, научился эффективно управлять буферами и сравнивать производительность языков при решении ресурсоёмких задач. Полученные знания могут быть применены при обработке данных, не помещающихся в оперативную память, например, в системах анализа логов или ETL-процессах.

Листинги кода

Листинг 1: Код модуля на C++

```
1  #include <cstring>
2  #include <fstream>
3  #include <vector>
4  #include <algorithm>
5  #include <filesystem>
6  #include <queue>
7  #include <string>
8  #include <iostream>
9
10 #include "external_sort.h"
11
12 using namespace std;
13 namespace fs = filesystem;
14
15
16 const size_t READER_SIZE = 1024*1024*10; // 10 MiB
17 const size_t MAX_RECORDS = 500000;
18 const size_t WRITER_SIZE = 1024*1024*10; // 10 MiB
19
20 const char TRUE[] = "true";
21 const char FALSE[] = "false";
22
23 struct __attribute__((packed)) Record {
24     char nickname[24];
25     char uuid[37];
26     char reg_date[11];
27     unsigned int level;
28     float hours;
29     bool vac_ban;
30 };
31
32 const size_t RECORD_SIZE = sizeof(Record);
33
```

```

34 struct MergeNode {
35     Record rec;
36     int fileIndex;
37 };
38
39
40 using Comparator = bool (*)(const Record&, const Record&);
41
42 static const Comparator comparators[2][6] = {
43     { // descending (по убыванию) - индекс 0
44         [](const Record& a, const Record& b) { return std::strcmp(a.
45             nickname, b.nickname) > 0; },
46         [](const Record& a, const Record& b) { return std::strcmp(a.
47             uuid, b.uuid) > 0; },
48         [](const Record& a, const Record& b) { return std::strcmp(a.
49             reg_date, b.reg_date) > 0; },
50         [](const Record& a, const Record& b) { return a.level > b.level
51             ; },
52         [](const Record& a, const Record& b) { return a.hours > b.hours
53             ; },
54         [](const Record& a, const Record& b) { return a.vac_ban > b.
55             vac_ban; }
56     },
57     { // ascending (по возрастанию) - индекс 1
58         [](const Record& a, const Record& b) { return std::strcmp(a.
59             nickname, b.nickname) < 0; },
60         [](const Record& a, const Record& b) { return std::strcmp(a.
61             uuid, b.uuid) < 0; },
62         [](const Record& a, const Record& b) { return std::strcmp(a.
63             reg_date, b.reg_date) < 0; },
64         [](const Record& a, const Record& b) { return a.level < b.level
65             ; },
66         [](const Record& a, const Record& b) { return a.hours < b.hours
67             ; },
68         [](const Record& a, const Record& b) { return a.vac_ban < b.
69             vac_ban; }
70     }
71 };

```

```

58     }
59 };
60
61
62 void parse(char **index, Record &rec) {
63     int i;
64
65     for (i = 0; (**index) != ','; i++, (*index)++)
66         rec.nickname[i] = (**index);
67     rec.nickname[i] = 0;
68     (*index)++;
69
70     for (i = 0; (**index) != ','; i++, (*index)++) {
71         // if ((**index) == ' ') continue;
72         rec.uuid[i] = (**index);
73     }
74     rec.uuid[i] = 0;
75     (*index)++;
76
77     for (i = 0; (**index) != ','; i++, (*index)++)
78         rec.reg_date[i] = (**index);
79     rec.reg_date[i] = 0;
80     (*index)++;
81
82     rec.level = strtol(*index, index, 10);
83     (*index)++;
84
85     rec.hours = strtod(*index, index);
86     (*index)++;
87
88     rec.vac_ban = false;
89     if (memcmp(*index, "true", 4) == 0)
90         rec.vac_ban = true;
91     while ((**index) != '\n') (*index)++;
92     (*index)++;
93 }

```

```

94
95
96 void serialize(char **index, Record &rec) {
97     int i;
98
99     for (i = 0; rec.nickname[i] != 0; i++, (*index)++)
100         (**index) = rec.nickname[i];
101     (**index) = ',';
102     (*index)++;
103
104     for (i = 0; rec.uuid[i] != 0; i++, (*index)++)
105         (**index) = rec.uuid[i];
106     (**index) = ',';
107     (*index)++;
108
109     for (i = 0; rec.reg_date[i] != 0; i++, (*index)++)
110         (**index) = rec.reg_date[i];
111     (**index) = ',';
112     (*index)++;
113
114     (*index) += sprintf(*index, "%d", rec.level);
115     (*index) += sprintf(*index, "%.3f", rec.hours);
116
117     const char *buf = rec.vac_ban ? TRUE : FALSE;
118     for (i = 0; buf[i] != 0; i++, (*index)++)
119         (**index) = buf[i];
120     (**index) = '\n';
121     (*index)++;
122 }
123
124
125 Export void external_sort(const char *path, int keyIndex, bool
    ascending, void (*progressCallback)(float)) {
126     ifstream input;
127     ofstream output;
128     Comparator cmp = comparators[ascending][keyIndex];

```

```

129 string tempDir = "temp";
130
131 fs::create_directories(tempDir);
132
133 input.open(path, std::ios::binary);
134 input.seekg(0, std::ios::end);
135 size_t totalFileSize = input.tellg();
136 double readIt = 0;
137 input.seekg(0, std::ios::beg);
138
139 size_t totalFiles = 0;
140 size_t totalRecords = 0;
141 char *buffer = new char[READER_SIZE+1];
142 char *index;
143 Record *records = new Record[MAX_RECORDS];
144
145 progressCallback(0);
146 while (1) {
147     // чтение куска файла.
148     input.read(buffer, READER_SIZE);
149     std::streamsize bytesRead = input.gcount();
150     if (bytesRead == 0) break;
151     // восстановление указателя
152     size_t currentFilePos = input.tellg();
153     index = buffer + (bytesRead-1);
154     for (; *index != '\n'; index--) // не очень безопасно
155         currentFilePos--;
156     *(index+1) = 0;
157     readIt += (index+1) - buffer;
158     input.seekg(currentFilePos, std::ios::beg);
159     // переход к чтению
160     index = buffer;
161     while (*index != 0) {
162         parse(&index, records[totalRecords]);
163         totalRecords++;
164         if (totalRecords == MAX_RECORDS) {

```

```

165         sort(records, records + totalRecords, cmp);
166         output.open(tempDir + "/r" + to_string(totalFiles++) + ".
            tmp", ios::binary);
167         output.write(reinterpret_cast<char*>(records), RECORD_SIZE
            * totalRecords);
168         output.close();
169         progressCallback(readIt / totalFileSize);
170         totalRecords = 0;
171     }
172 }
173 }
174 input.close();
175
176 if (totalRecords > 0) {
177     sort(records, records + totalRecords, cmp);
178     output.open(tempDir + "/r" + to_string(totalFiles++) + ".tmp",
        ios::binary);
179     output.write(reinterpret_cast<char*>(records), RECORD_SIZE *
        totalRecords);
180     output.close();
181     progressCallback(readIt / totalFileSize);
182 }
183
184 delete[] buffer;
185 delete[] records;
186
187 progressCallback(0);
188 auto inverted_cmp = [&](const MergeNode& a, const MergeNode& b) {
189     return cmp(b.rec, a.rec);
190 };
191
192 priority_queue<MergeNode, vector<MergeNode>, decltype(
    inverted_cmp)> pq(inverted_cmp);
193 vector<ifstream*> openFiles;
194 output.open(string(path) + ".sorted", ios::binary);
195

```

```

196 for (int i = 0; i < totalFiles; ++i) {
197     auto* file = new ifstream(tempDir + "/" + to_string(i) + ".tmp", ios::binary);
198     Record rec;
199     file->read((char*)&rec, RECORD_SIZE);
200     if (file->gcount() == RECORD_SIZE) {
201         pq.push({rec, i});
202     }
203     openFiles.push_back(file);
204 }
205
206 readIt = 0;
207 buffer = new char[WRITER_SIZE];
208 index = buffer;
209 size_t size;
210
211 while (!pq.empty()) {
212     MergeNode top = pq.top();
213     pq.pop();
214     serialize(&index, top.rec);
215     size = index - buffer;
216     if (size > WRITER_SIZE-1000) {
217         output.write(buffer, size);
218         readIt += size;
219         progressCallback(readIt / totalFileSize);
220         index = buffer;
221     }
222
223     Record rec;
224     openFiles[top.fileIndex]->read((char*)&rec, RECORD_SIZE);
225     if (openFiles[top.fileIndex]->gcount() == RECORD_SIZE) {
226         pq.push({rec, top.fileIndex});
227     }
228 }
229
230 if (index > buffer) {

```

```

231     output.write(buffer, size);
232     readIt += size;
233     progressCallback(readIt / totalFileSize);
234 }
235
236 delete[] buffer;
237 for (auto file : openFiles) { file->close(); delete file; }
238 fs::remove_all(tempDir);
239 output.close();
240 }
241
242
243 int main() {
244     auto progress = [](float p) {
245         printf("Прогресс: %.2f%%\n", p * 100);
246     };
247
248     // ascending (по возрастанию = true)
249     // 0 nickname
250     // 1 uuid
251     // 2 reg_date
252     // 3 level
253     // 4 hours
254     // 5 vac_ban
255     external_sort("data.csv", 2, false, progress);
256     return 0;
257 }
258
259 // как приложение:
260 // g++ external_sort.cpp -o ext
261 // как библиотеку:
262 // g++ -shared -o external_sort.dll external_sort.cpp -static -Os -
    s

```


Листинг 2: Заголовочный файл для C++

```
1 #pragma once
2 #define Export __declspec(dllexport)
3
4 extern "C" {
5     Export void external_sort(
6         const char *path,
7         int keyIndex,
8         bool ascending,
9         void (*progressCallback)(float)
10    );
11 }
```

Листинг 3: Код модуля на Python

```
1 import os
2 import struct
3 import heapq
4 import shutil
5 from pathlib import Path
6 import numpy as np
7 import gc
8 from typing import Callable
9
10 # Константы (в соответствии с C++ версией)
11 READER_SIZE = 1024 * 1024 * 10 # 10 MiB - буфер чтения
12 MAX_RECORDS = 500_000
13 WRITER_SIZE = 1024 * 1024 * 10 # 10 MiB - буфер записи
14
15 # Определяем numpy тип данных (соответствует C++ __attribute__((
16     packed))) Record)
17 # Строковые поля (S) фиксированной длины, little-endian типы для
18     чисел
19 record_dtype = np.dtype([
20     ('nickname', 'S24'),
21     ('uuid', 'S37'),
22     ('reg_date', 'S11'),
```

```

21     ('level', '<u4'),
22     ('hours', '<f4'),
23     ('vac_ban', 'b1')
24 ], align=False) # align=False гарантирует плотную упаковку ровно в
    81 байт
25
26 class MergeNode:
27     """Узел для приоритетной очереди в фазе слияния"""
28     __slots__ = ['key', 'file_index', 'raw_bytes', 'ascending']
29
30     def __init__(self, key, file_index: int, raw_bytes: bytes,
31                   ascending: bool):
32         self.key = key
33         self.file_index = file_index
34         self.raw_bytes = raw_bytes
35         self.ascending = ascending
36
37     def __lt__(self, other):
38         # Если ascending == False, инвертируем оператор сравнения, как
39         # в C++ лямбде
40         if self.ascending:
41             return self.key < other.key
42         return self.key > other.key
43
44     def extract_key(raw_bytes: bytes, keyIndex: int):
45         """
46         Извлечение ключа для сортировки из 81-байтной записи
47         Учитывает обрезку нулевых байтов (\\x00) для строковых полей.
48         """
49         if keyIndex == 0:
50             return raw_bytes[0:24].split(b'\\x00', 1)[0]
51         elif keyIndex == 1:
52             return raw_bytes[24:61].split(b'\\x00', 1)[0]
53         elif keyIndex == 2:
54             return raw_bytes[61:72].split(b'\\x00', 1)[0]
55         elif keyIndex == 3:

```

```

54     return struct.unpack('<I', raw_bytes[72:76])[0]
55 elif keyIndex == 4:
56     return struct.unpack('<f', raw_bytes[76:80])[0]
57 elif keyIndex == 5:
58     return raw_bytes[80] != 0
59
60 def _sort_and_dump(records: np.ndarray, total: int, keyIndex: int,
61     ascending: bool, temp_dir: Path, file_idx: int):
62     """Сортировка текущего блока numpy массива на месте и сохранение
63     в бинарный файл"""
64     fields = ['nickname', 'uuid', 'reg_date', 'level', 'hours', '
65         vac_ban']
66     sort_field = fields[keyIndex]
67
68     # Берем срез (view) реально заполненных данных
69     view = records[:total]
70
71     # Сортировка O(N log N) на уровне C-библиотек Numpy
72     view.sort(order=sort_field)
73
74     # Если убывание, просто разворачиваем view за O(1) память, не
75     копируя данные
76     if not ascending:
77         view = view[::-1]
78
79     out_path = temp_dir / f"r{file_idx}.tmp"
80     view.tofile(out_path) # Сброс бинарного дампа
81
82 def external_sort(path: bytes, keyIndex: int, ascending: bool,
83     progressCallback: Callable[[float], None]) -> None:
84     path = path.decode()
85     temp_dir = Path("temp")
86     if temp_dir.exists():
87         shutil.rmtree(temp_dir)
88     temp_dir.mkdir(parents=True, exist_ok=True)

```

```

85 total_file_size = os.path.getsize(path)
86 if total_file_size == 0:
87     return
88
89 # Фаза 1: Блочное Чтение и Сортировка (Разделение)
90 records = np.empty(MAX_RECORDS, dtype=record_dtype)
91 reader_buf = bytearray(READER_SIZE)
92
93 total_files = 0
94 total_records = 0
95 read_it = 0
96
97 progressCallback(0.0)
98
99 with open(path, "rb") as f_in:
100     while True:
101         # Чтение напрямую в предвыделенный буфер для экономии памяти
102         bytes_read = f_in.readinto(reader_buf)
103         if bytes_read == 0:
104             break
105
106         # Поиск последнего перевода строки, чтобы не разрывать запись
107         last_newline = reader_buf.rfind(b'\n', 0, bytes_read)
108         if last_newline == -1:
109             last_newline = bytes_read - 1
110
111         # Откат файлового курсора до неполной строки
112         f_in.seek(last_newline + 1 - bytes_read, os.SEEK_CUR)
113         read_it += (last_newline + 1)
114
115         # Парсинг текущего 10 МБ буфера байтовыми операциями
116         idx = 0
117         while idx <= last_newline:
118             if total_records == MAX_RECORDS:
119                 _sort_and_dump(records, total_records, keyIndex,
                                ascending, temp_dir, total_files)

```

```

120         total_files += 1
121         total_records = 0
122         progressCallback(read_it / total_file_size)
123
124         # Достигнут конец безопасного участка
125         if idx == last_newline + 1:
126             break
127
128         # Ручной парсинг индексов по запятой
129         comma1 = reader_buf.find(b',', idx, last_newline + 1)
130         if comma1 == -1:
131             break
132         records['nickname'][total_records] = bytes(reader_buf[idx:
133             comma1])
134
135         comma2 = reader_buf.find(b',', comma1 + 1, last_newline +
136             1)
137         records['uuid'][total_records] = bytes(reader_buf[comma1+1:
138             comma2])
139
140         comma3 = reader_buf.find(b',', comma2 + 1, last_newline +
141             1)
142         records['reg_date'][total_records] = bytes(reader_buf[
143             comma2+1:comma3])
144
145         comma4 = reader_buf.find(b',', comma3 + 1, last_newline +
146             1)
147         records['level'][total_records] = int(reader_buf[comma3+1:
148             comma4])
149
150         comma5 = reader_buf.find(b',', comma4 + 1, last_newline +
151             1)
152         records['hours'][total_records] = float(reader_buf[comma4
153             +1:comma5])
154
155         newline = reader_buf.find(b'\n', comma5 + 1, last_newline +

```

```

1)
147     if newline == -1:
148         newline = last_newline
149         # Учтем возможный \r от Windows (если файл формата CRLF)
150         records['vac_ban'][total_records] = reader_buf[comma5+1:
151             newline].startswith(b'true')
152
153     idx = newline + 1
154     total_records += 1
155
156     # Запись последнего недозаполненного блока
157     if total_records > 0:
158         _sort_and_dump(records, total_records, keyIndex, ascending,
159             temp_dir, total_files)
160         total_files += 1
161         progressCallback(read_it / total_file_size)
162
163     # ОСВОБОЖДЕНИЕ ПАМЯТИ
164     del records
165     del reader_buf
166     gc.collect()
167
168     progressCallback(0.0)
169
170     # Фаза 2: K-Way Merge (Слияние)
171     # Открываем все временные файлы для чтения
172     open_files = []
173     pq = []
174
175     for i in range(total_files):
176         f = open(temp_dir / f"r{i}.tmp", "rb")
177         raw_bytes = f.read(81) # Читаем ровно 1 структуру (81 байт)
178         if len(raw_bytes) == 81:
179             key = extract_key(raw_bytes, keyIndex)
180             heapq.heappush(pq, MergeNode(key, i, raw_bytes, ascending))
181         open_files.append(f)

```

```

180
181 writer_buf = bytearray(WRITER_SIZE)
182 writer_pos = 0
183 read_out_size = 0
184
185 with open(f"{path}.sorted", "wb") as f_out:
186     while pq:
187         node = heapq.heappop(pq)
188         raw_bytes = node.raw_bytes
189         f_idx = node.file_index
190
191         # Десериализация (парсим бинарные данные 81 байт)
192         nick = raw_bytes[0:24].split(b'\x00', 1)[0]
193         uuid = raw_bytes[24:61].split(b'\x00', 1)[0]
194         date = raw_bytes[61:72].split(b'\x00', 1)[0]
195         level = struct.unpack('<I', raw_bytes[72:76])[0]
196         hours = struct.unpack('<f', raw_bytes[76:80])[0]
197         ban_str = b"true" if raw_bytes[80] != 0 else b"false"
198
199         # Векторное форматирование байт (% оператор для bytes очень
200             # быстр и не создает Unicode-строку)
201         row = b"%b,%b,%b,%d,%.3f,%b\n" % (nick, uuid, date, level,
202             hours, ban_str)
203         row_len = len(row)
204
205         # Запись в буфер слияния
206         writer_buf[writer_pos:writer_pos+row_len] = row
207         writer_pos += row_len
208         read_out_size += row_len
209
210         # Сброс буфера (по аналогии с C++: WRITER_SIZE - 1000)
211         if writer_pos > WRITER_SIZE - 1000:
212             f_out.write(memoryview(writer_buf)[:writer_pos])
213             progressCallback(read_out_size / total_file_size)
214             writer_pos = 0

```

```

214     # Чтение следующей записи из того же файла, откуда была взята
        минимальная
215     next_bytes = open_files[f_idx].read(81)
216     if len(next_bytes) == 81:
217         node.key = extract_key(next_bytes, keyIndex)
218         node.raw_bytes = next_bytes
219         heapq.heappush(pq, node)
220
221     # Сброс остатков буфера
222     if writer_pos > 0:
223         f_out.write(memoryview(writer_buf)[:writer_pos])
224         read_out_size += writer_pos
225         p: float = read_out_size / total_file_size
226         if p > 1.0:
227             p = 1.0
228         progressCallback(p)
229
230     # Очистка ресурсов
231     for f in open_files:
232         f.close()
233     shutil.rmtree(temp_dir)
234
235 if __name__ == "__main__":
236     def progress(p: float):
237         print(f"Прогресс: {p * 100:.2f}%")
238
239     # Сортировка файла: path, keyIndex, ascending, callback
240     # Индексы:
241     # 0 = nickname
242     # 1 = uuid
243     # 2 = reg_date
244     # 3 = level
245     # 4 = hours
246     # 5 = vac_ban
247     external_sort("data.csv", 0, True, progress)

```


Листинг 4: Код генератора данных

```
1
2 from time import time
3 from datetime import datetime
4 from random import random, choice, randint
5 import math
6 import uuid
7
8 nicks = {
9     # 23 коротких ника (однословные, 5-7 символов)
10    "Raven": [],
11    "Tiger": [],
12    "Falcon": [],
13    "Viper": [],
14    "Eagle": [],
15    "Wolver": [],
16    "Lynx": [],
17    "Hawkeye": [],
18    "Crow": [],
19    "Fox": [],
20    "Bear": [],
21    "Lion": [],
22    "Boar": [],
23    "Elk": [],
24    "Owl": [],
25    "Bat": [],
26    "Seal": [],
27    "Crab": [],
28    "Wasp": [],
29    "Ant": [],
30    "Beetle": [],
31    "Fly": [],
32    "Spider": [],
33
34    # 40 длинных ников с дополнительной конструкцией (два слова +
    суффикс)
```

```
35 "ShadowStalkerXxX": [],
36 "NightCrawlerGG": [],
37 "FireBreatherF": [],
38 "StormWeaverAAA": [],
39 "EarthShakerZzZ": [],
40 "WindWhisperQwQ": [],
41 "ThunderMasterOoO": [],
42 "LightningStrikeBz": [],
43 "CrystalGuardKk": [],
44 "DarkHunterVvV": [],
45 "LightBringerMew": [],
46 "SoulKeeperNyx": [],
47 "MindReaderRz": [],
48 "TimeWalkerPw": [],
49 "SpaceRiderLol": [],
50 "StarGazerWwW": [],
51 "MoonDancerHhH": [],
52 "SunChaserYyY": [],
53 "DawnBreakerPp": [],
54 "DuskTreaderRr": [],
55 "FrostBiteXxX": [],
56 "FlameBringerGG": [],
57 "EmberSparkF": [],
58 "BlazeFuryAAA": [],
59 "VortexMasterZzZ": [],
60 "PhantomGhostQwQ": [],
61 "ShadowReaperOoO": [],
62 "SilentAssassinBz": [],
63 "StealthNinjaKk": [],
64 "QuickSilverVvV": [],
65 "IronFistMew": [],
66 "SteelHeartNyx": [],
67 "BronzeShieldRz": [],
68 "GoldenEaglePw": [],
69 "SilverWolfLol": [],
70 "CrimsonFoxWwW": [],
```

```

71 "EmeraldDragonHhH": [],
72 "SapphireTigerYyY": [],
73 "RubyLionPp": [],
74 "DiamondHawkRr": [],
75
76 # 37 длинных ников без конструкции (просто два слова)
77 "ObsidianSerpent": [],
78 "GraniteGolem": [],
79 "MarbleMystic": [],
80 "QuartzKnight": [],
81 "TopazTitan": [],
82 "JadeJaguar": [],
83 "AmberApe": [],
84 "CoralCobra": [],
85 "PearlPegasus": [],
86 "RubyRaven": [],
87 "SapphireSphinx": [],
88 "TopazTroll": [],
89 "ObsidianOwl": [],
90 "GraniteGriffin": [],
91 "MarbleManticore": [],
92 "QuartzQuetzal": [],
93 "JadeJinni": [],
94 "AmberArchon": [],
95 "CoralCentaur": [],
96 "PearlPhoenix": [],
97 "RubyRogue": [],
98 "SapphireSorcerer": [],
99 "TopazTamer": [],
100 "ObsidianOracle": [],
101 "GraniteGuardian": [],
102 "MarbleMage": [],
103 "QuartzQueen": [],
104 "JadeJester": [],
105 "AmberArcher": [],
106 "CoralCrusader": [],

```

```

107     "OnyxOutcast": [],
108     "PearlPaladin": [],
109     "RubyRanger": [],
110     "SapphireShaman": [],
111     "TopazThief": [],
112     "ObsidianOverseer": [],
113     "OnyxWarden": []
114 }
115
116
117 def atan_transform(x, threshold=40000, max_value=50000):
118     if x <= threshold:
119         return x
120     else:
121         # Масштабируем вход для atan, чтобы в threshold получить нужный
122         # наклон
123         k = 2 / threshold # коэффициент масштабирования
124         # atan стремится к  $\pi/2$ , поэтому масштабируем выход
125         return threshold + (max_value - threshold) * (2 / math.pi) *
126             math.atan(k * (x - threshold))
127
128
129 def gen_date_and_hours():
130     x = chunk_time * random()
131     y = (chunk_time - x) * random() * 0.8 / 3600
132     dt = datetime.fromtimestamp(x + start_time)
133     return dt.strftime('%Y.%m.%d'), f"{atan_transform(y):.3f}"
134
135
136 def gen_line():
137     nick = choice(list(nicks.keys()))
138     date, hours = gen_date_and_hours()
139     level = str(randint(0, 100))
140     ban = ("true" if randint(0, 99) <= 4 else "false")
141     id = str(uuid.uuid5(uuid.NAMESPACE_DNS, ','.join([nick, date,
142         level, hours, ban])))

```

```

140     return ",".join([nick, id, date, level, hours, ban]) + "\n"
141
142
143 start_time = int(datetime(2000, 1, 1).timestamp())
144 end_time = int(datetime.now().timestamp())
145 chunk_time = end_time - start_time
146
147
148 def generate(path: str, size_max: int, func = None) -> None:
149     with open(path, "wb") as file:
150         file.seek(0)
151         chunk = size_max / 100
152         counter = 0
153         size = 0
154         while size < size_max:
155             if size > chunk*counter:
156                 p = size / size_max
157                 if p > 1: p = 1.0
158                 if func is not None: func(p)
159                 counter+=1
160                 line = gen_line()
161                 file.write(line.encode("ascii"))
162                 file.flush()
163                 size += len(line)
164
165 # файл data.csv через запятую
166 # 1 строка: ник
167 # 2 строка: айди (uuid)
168 # 3 строка: дата регистрации (начиная с 2000.01.01)
169 # 4 инт: уровень (0-100)
170 # 5 флот: кол-во часов (меньше или равно с момента регистрации)
171 # 6 бул: вак бан?
172
173 if __name__ == "__main__":
174     generate("data.csv", (1024**3)*1.05)

```

Листинг 5: Код приложения с пользовательским интерфейсом

```
1 import tkinter as tk
2 from tkinter import ttk, filedialog, messagebox
3 from threading import Thread
4 import csv
5 import os
6 import ctypes
7
8 # Импорт локального модуля
9 try:
10     import generate
11 except ImportError:
12     generate = None
13
14 class App(tk.Tk):
15     def __init__(self):
16         super().__init__()
17
18         self.title("CSV Processor & External Sorter")
19         self.geometry("1150x650")
20         self.configure(bg="#f0f0f0")
21
22         # Переменные состояния
23         self.file_path = tk.StringVar(value="")
24         self.num_lines_in_file = tk.IntVar(value=0)
25
26         # Загрузка DLL
27         self.mod1 = None
28         if os.path.exists("external_sort.dll"):
29             try:
30                 self.mod1 = ctypes.CDLL("./external_sort.dll")
31                 self.mod1.external_sort.argtypes = [
32                     ctypes.c_char_p,
33                     ctypes.c_int,
34                     ctypes.c_bool,
35                     ctypes.CFUNCTYPE(None, ctypes.c_float)
```

```

36         ]
37         self.mod1.external_sort.restype = None
38     except Exception as e:
39         print(f"Ошибка загрузки DLL: {e}")
40
41     import external_sort as mod2
42     self.mod2 = mod2
43
44     self.lib = self.mod1
45     self.init_ui()
46
47 def init_ui(self):
48     # --- Сверху: Прогресс-бар ---
49     style = ttk.Style()
50     style.theme_use('default')
51     style.configure("Green.Horizontal.TProgressbar", foreground='
        #28a745', background='#28a745')
52
53     separator_style = ttk.Style()
54     separator_style.configure("Purple.TSeparator", background="#
        c72dbf")
55
56     self.progress = ttk.Progressbar(self, orient="horizontal",
57                                     length=100,
58                                     mode="determinate", style="Green.Horizontal.
59                                         TProgressbar")
60     self.progress.pack(side="top", fill="x")
61
62     # Основной контейнер
63     main_container = tk.Frame(self, bg="#f0f0f0")
64     main_container.pack(side="top", fill="both", expand=True, padx
65                         =10, pady=10)
66
67     # --- Левая часть ---
68     self.left_panel = tk.Frame(main_container, bg="#f0f0f0")
69     self.left_panel.pack(side="left", fill="y", padx=(0, 10))

```

```

67
68 # 1. Файл и путь
69 file_info_frame = tk.Frame(self.left_panel, bg="#f0f0f0")
70 file_info_frame.pack(fill="x", pady=5)
71
72 tk.Label(file_info_frame, text="Файл: ", font=("Arial", 9, "
    bold"), bg="#f0f0f0").pack(side="top", anchor="w")
73 self.path_label = tk.Label(file_info_frame, text="Не выбран",
    fg="red",
74                             wraplength=220, justify="left", bg="#f0f0f0")
75 self.path_label.pack(side="top", anchor="w")
76
77 # Лейбл для отображения количества строк
78 self.lines_count_label = tk.Label(file_info_frame, text="Кол-во
    строки: 0",
79                                     font=("Arial", 9), fg="#555", bg="#f0f0f0")
80 self.lines_count_label.pack(side="top", anchor="w", pady=(2, 0)
    )
81
82 # 2. Кнопка выбора
83 tk.Button(self.left_panel, text="Выбрать файл", command=self.
    select_file).pack(fill="x", pady=5)
84
85 ttk.Separator(self.left_panel, orient="horizontal", style="
    Purple.TSeparator").pack(fill="x", pady=10)
86
87 # 3. Поле для ввода числа (1-2048)
88 tk.Label(self.left_panel, text="Размер (1-2048):", bg="#f0f0f0"
    ).pack(anchor="w")
89 vcmd = (self.register(self.validate_int), '%P')
90 self.size_entry = tk.Entry(self.left_panel, validate='key',
    state='disabled', validatecommand=vcmd)
91 self.size_entry.insert(0, "1")
92 self.size_entry.pack(fill="x", pady=2)
93
94 # 4. Кнопка Сгенерировать

```



```

95 def gen_thread():
96     Thread(target=self.run_generate, daemon=True).start()
97 self.gen_button = tk.Button(self.left_panel, text="
98     Сгенерировать", state="disabled", command=gen_thread)
99 self.gen_button.pack(fill="x", pady=5)
100
101 ttk.Separator(self.left_panel, orient="horizontal", style="
102     Purple.TSeparator").pack(fill="x", pady=10)
103
104 # Модуль
105 tk.Label(self.left_panel, text="Модуль:", bg="#f0f0f0").pack(
106     anchor="w")
107 self.mod_options = ["C++", "Python"]
108 self.mod_combo = ttk.Combobox(self.left_panel, values=self.
109     mod_options, state="disabled")
110 self.mod_combo.current(0)
111 self.mod_combo.pack(fill="x", pady=2)
112 self.mod_combo.bind("<<ComboboxSelected>>", self.on_mod_combo)
113
114 # 5. Ключ
115 tk.Label(self.left_panel, text="Ключ:", bg="#f0f0f0").pack(
116     anchor="w")
117 self.key_options = ["Ник", "Айди", "Дата регистрации", "Уровень
118     ", "Кол-во часов", "Вак бан"]
119 self.key_combo = ttk.Combobox(self.left_panel, values=self.
120     key_options, state="disabled")
121 self.key_combo.current(0)
122 self.key_combo.pack(fill="x", pady=2)
123
124 # 6. Направление
125 tk.Label(self.left_panel, text="Направление:", bg="#f0f0f0").
126     pack(anchor="w")
127 self.dir_options = ["По убыванию", "По возрастанию"]
128 self.dir_combo = ttk.Combobox(self.left_panel, values=self.
129     dir_options, state="disabled")
130 self.dir_combo.current(1)

```

```

122 self.dir_combo.pack(fill="x", pady=2)
123
124 # 7. Кнопка Сортировать
125 def sort_thread():
126     Thread(target=self.run_sort, daemon=True).start()
127 self.sort_btn = tk.Button(self.left_panel, text="Сортировать",
128     state="disabled", command=sort_thread)
129 self.sort_btn.pack(fill="x", pady=5)
130
131 ttk.Separator(self.left_panel, orient="horizontal", style="
132     Purple.TSeparator").pack(fill="x", pady=10)
133
134 # 8-10. Настройки отображения
135 self.file_target = tk.StringVar(value="orig")
136 tk.Radiobutton(self.left_panel, text="Исходный файл", variable=
137     self.file_target,
138     value="orig", bg="#f0f0f0").pack(anchor="w")
139 tk.Radiobutton(self.left_panel, text="Сортированный файл",
140     variable=self.file_target,
141     value="sort", bg="#f0f0f0").pack(anchor="w")
142
143 tk.Label(self.left_panel, text="Начать со строки:", bg="#f0f0f0
144     ").pack(anchor="w")
145 self.start_line_entry = tk.Entry(self.left_panel)
146 self.start_line_entry.insert(0, "0")
147 self.start_line_entry.pack(fill="x", pady=2)
148
149 tk.Label(self.left_panel, text="Кол-во строк:", bg="#f0f0f0").
150     pack(anchor="w")
151 self.count_line_entry = tk.Entry(self.left_panel)
152 self.count_line_entry.insert(0, "10")
153 self.count_line_entry.pack(fill="x", pady=2)
154
155 # 11. Кнопка Отобразить
156 tk.Button(self.left_panel, text="Отобразить", command=self.
157     display_content).pack(fill="x", pady=5)

```

```

151
152 # --- Правая часть: Output ---
153 right_panel = tk.Frame(main_container)
154 right_panel.pack(side="right", fill="both", expand=True)
155
156 # Ширина ровно 100 символов
157 self.output_text = tk.Text(right_panel, width=100, wrap="none",
158                             state="disabled", font=("Courier New", 10))
159 scroll_y = tk.Scrollbar(right_panel, orient="vertical", command
160                        =self.output_text.yview)
161 scroll_x = tk.Scrollbar(right_panel, orient="horizontal",
162                        command=self.output_text.xview)
163
164 self.output_text.configure(yscrollcommand=scroll_y.set,
165                             xscrollcommand=scroll_x.set)
166
167 scroll_y.pack(side="right", fill="y")
168 scroll_x.pack(side="bottom", fill="x")
169 self.output_text.pack(side="left", fill="both", expand=True)
170
171 # --- Логика ---
172
173 def on_mod_combo(self, event):
174     match self.mod_combo.current():
175         case 0:
176             self.lib = self.mod1
177         case 1:
178             self.lib = self.mod2
179
180 def validate_int(self, P):
181     if P == "":
182         return True
183     try:
184         val = int(P)
185         return 1 <= val <= 2048
186     except ValueError:

```

```

183         return False
184
185     def update_progress(self, value):
186         self.progress['value'] = value * 100
187         self.update_idletasks()
188
189     def select_file(self):
190         path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv")])
191
192         if path:
193             self.file_path.set(path)
194             self.path_label.config(text=path, fg="gray")
195
196             # Активируем элементы
197             self.mod_combo.config(state="readonly")
198             self.key_combo.config(state="readonly")
199             self.dir_combo.config(state="readonly")
200             self.sort_btn.config(state="normal")
201
202             self.size_entry.config(state='normal')
203             self.gen_button.config(state='normal')
204
205             # Подсчет строк
206             self.count_total_lines(path)
207         else:
208             # self.path_label.config(text="Не выбран", fg="red")
209             # self.lines_count_label.config(text="Кол-во строк: 0")
210             # self.sort_btn.config(state="disabled")
211
212     def count_total_lines(self, path):
213         try:
214             with open(path, 'r', encoding='utf-8') as f:
215                 # Более надежный способ для CSV
216                 count = sum(1 for _ in f)
217                 self.num_lines_in_file.set(count)

```

```

218         self.lines_count_label.config(text=f"Кол-во строк: {count}"
219                                         )
220     except Exception as e:
221         messagebox.showerror("Ошибка", f"Не удалось прочитать файл: {
222             e}")
223
224 def run_generate(self):
225     self.sort_btn.config(state='disabled')
226     self.gen_button.config(state='disabled')
227     if not self.file_path.get():
228         messagebox.showwarning("Внимание", "Сначала выберите файл!")
229         self.sort_btn.config(state='normal')
230         self.gen_button.config(state='normal')
231         return
232
233     if generate:
234         size_bytes = int(self.size_entry.get()) * (1024**2)
235         generate.generate(self.file_path.get(), size_bytes, self.
236                         update_progress)
237         self.count_total_lines(self.file_path.get())
238         messagebox.showinfo("Готово", "Генерация завершена")
239     else:
240         messagebox.showerror("Ошибка", "Модуль 'generate' не найден")
241     self.sort_btn.config(state='normal')
242     self.gen_button.config(state='normal')
243
244 def run_sort(self):
245     self.sort_btn.config(state='disabled')
246     self.gen_button.config(state='disabled')
247     path = self.file_path.get()
248     # Проверка на \n в конце
249     try:
250         with open(path, 'rb+') as f:
251             f.seek(-1, os.SEEK_END)
252             if f.read(1) != b'\n':
253                 messagebox.showerror("Ошибка", "Файл должен заканчиваться

```

```

        символом переноса строки (LF).")
251         self.sort_btn.config(state='normal')
252         self.gen_button.config(state='normal')
253         return
254     except Exception as e:
255         messagebox.showerror("Ошибка доступа", str(e))
256         self.sort_btn.config(state='normal')
257         self.gen_button.config(state='normal')
258         return
259
260     if not self.lib:
261         messagebox.showerror("Ошибка", "DLL не загружена")
262         self.sort_btn.config(state='normal')
263         self.gen_button.config(state='normal')
264         return
265
266     CMPFUNC = ctypes.CFUNCTYPE(None, ctypes.c_float)
267     callback_c = CMPFUNC(self.update_progress)
268
269     key_idx = self.key_combo.current()
270     ascending = bool(self.dir_combo.current())
271
272     try:
273         self.lib.external_sort(path.encode('utf-8'), key_idx,
274                                ascending, callback_c)
275         messagebox.showinfo("Успех", "Сортировка завершена")
276     except Exception as e:
277         messagebox.showerror("Ошибка DLL", str(e))
278         self.sort_btn.config(state='normal')
279         self.gen_button.config(state='normal')
280
281     def display_content(self):
282         base_path = self.file_path.get()
283         if not base_path:
284             return

```

```

285     target = base_path if self.file_target.get() == "orig" else
        base_path + ".sorted"
286
287     if not os.path.exists(target):
288         messagebox.showerror("Ошибка", f"Файл {target} не найден")
289         return
290
291     try:
292         start_line = int(self.start_line_entry.get())
293         count = int(self.count_line_entry.get())
294
295         self.output_text.config(state="normal")
296         self.output_text.delete("1.0", tk.END)
297
298         with open(target, 'r', encoding='utf-8') as f:
299             reader = csv.reader(f)
300             for i, row in enumerate(reader):
301                 if i < start_line:
302                     continue
303                 if i >= start_line + count:
304                     break
305                 self.output_text.insert(tk.END, ",".join(row) + "\n")
306
307         self.output_text.config(state="disabled")
308
309     except ValueError:
310         messagebox.showerror("Ошибка", "Введите корректные числа")
311
312 if __name__ == "__main__":
313     app = App()
314     app.mainloop()

```